

EiraNova — Eierhåndbok og opplæring

EiraNova-plattformen — Eierhåndbok og opplæring

For Lise og Jeanett. Skrevet for dere som ikke er teknikere, men som skal eie, forstå og styre plattformen selv. Målet: etter fire korte økter kan dere gjøre enkle endringer på egen hånd, bestille større ting presist, og aldri være avhengige av én person.

Hvorfor dette er verdt tiden

Alt dere bygger videre i selskapet, eier dere sammen — automatisk. Loven (åndsverkloven § 71) sier at programvare laget av ansatte i arbeidet tilhører arbeidsgiveren, altså EiraNova AS, altså dere to gjennom aksjene. Hver kontrakt dere gjennomfører selv, er verdi dere skaper sammen uten konsulentregning. Og kunnskapen om hvordan plattformen fungerer, er like mye deres begge — den er det ingen som kan ta fra dere.

Grunnlaget (det som finnes i dag) er en gave fra Richard til Lise. Det dere bygger fra nå av, er deres felles. Over tid blir det dere har laget selv den største delen.

Del 1 — Kartet: hva plattformen består av

Tenk på det som et hus med fem rom. Dere trenger ikke kunne bygge huset, men dere må vite hvor rommene er.

- **GitHub** = arkivet. Her ligger all kode («repoet» eiranova-platform), all dokumentasjon (mappen docs/) og hele historikken over hva som er gjort. Ingenting endres uten at det er synlig her.
- **Vercel** = det som er på nett. Vercel tar koden fra GitHub og gjør den til nettsider: eiranova.no (marketing), app.eiranova.no (kunde), nurse.eiranova.no (sykepleier), admin.eiranova.no (admin) og oppstart-appen.
- **Supabase** = databasen. Her lagres kunder, oppdrag, tjenester og innlogginger. Ligger i Frankfurt (EU). Dette er selskapets mest sensitive rom — her går man forsiktig.
- **Resend** = e-postutsendelse. Kontaktskjemaet og fremtidige varsler sendes herfra, fra post@eiranova.no.
- **Domeneshop** = adressen. Her eier dere eiranova.no og bestemmer hvor adressen peker.

Alle fem eies av selskapet (post@eiranova.no er eier). Passord og nøkler ligger i selskapets passordhvelv. Dere skal begge kunne logge inn overalt.

Del 2 — Metoden: slik gjøres alle endringer

Dere skriver verken kode eller kontrakter. Dere beskriver ønsket i vanlig norsk. To KI-verktøy gjør resten, med hver sin rolle:

- **Claude** = arkitekten. Kjenner plattformen (dere gir den en fersk kopi av repoet), oversetter ønsket deres til en presis kontrakt, og kontrollerer resultatet etterpå.
- **Cursor** = byggeren. Får kontrakten, gjør endringen i koden, tester, og lager forslaget (PR) i GitHub.
- **Dere** = eierne. Beskriver ønsket, ser på forhåndsvisningen, og godkjenner.

Slik ser hele løpet ut, hver gang:

1. **Ønske:** dere skriver til Claude i EiraNova-prosjektet: «Vi vil at prissiden skal vise de nye prisene: ...»
2. **Fersk kopi:** dere laster opp en ny zip av repoet (Del 3 forklarer hvordan). Uten den gjetter Claude — med den treffer den.
3. **Kontrakt:** Claude lager en ferdig K-kontrakt tilpasset nøyaktig slik koden ser ut nå. Dere leser den — den er skrevet så dere forstår hva som skal skje.
4. **Cursor bygger:** dere limer kontrakten inn i Cursor, som gjør endringen, tester og lager en PR. Cursor rapporterer når den er ferdig.
5. **Tilbake til Claude:** dere limer Cursors rapport inn i samme Claude-dialog. Claude sjekker at alt i kontrakten er gjort, og sier fra om noe mangler.
6. **Forhåndsvisning:** Vercel har laget en testversjon på egen adresse. Dere ser på den i nettleseren.
7. **Godkjenning:** ser det riktig ut, sier dere «godkjent» til Cursor, som sender endringen til dev og main.
8. **Live:** Vercel oppdaterer den ekte siden av seg selv. Claude noterer kontrakten som ferdig i køen.

Fem regler som aldri brytes:

- Én kontrakt om gangen. Ferdig med én før neste starter.
- Alltid fersk zip til Claude før en ny kontrakt. En kontrakt basert på gammel kode gir feil.
- Cursors rapport limes alltid tilbake til Claude — det er kvalitetskontrollen.
- Aldri endre direkte på main (det som er live). Alltid via PR og forhåndsvisning.
- Se alltid på forhåndsvisningen før dere godkjenner.
- Rapporten fra Cursor leses — den forteller hva som ble gjort og hva som ble sjekket.
- Er dere i tvil: stopp, og spør. Ingenting haster så mye at det er verdt å ødelegge noe.

Del 3 — Kom i gang (gjøres én gang, ca. 1 time)

- Opprett egen GitHub-bruker med @eiranova.no-adressen din. Be om å bli lagt til i organisasjonen eiranova.no.
- Claude: logg inn på claude.ai med selskapets konto (abonnement betales av selskapet). Bruk prosjektet «EiraNova» — der ligger metodikken, roadmapen og eierhåndboken som bakgrunnskunnskap, slik at Claude alltid vet hvordan dere jobber. Hver kontrakt er én ny samtale i prosjektet.
- Slik lager dere zip av repoet: gå til github.com/eiranova-no/eiranova-platform i nettleseren → velg branch «dev» øverst til venstre → grønn «Code»-knapp → «Down-

load ZIP». Last opp zip-filen i Claude-samtalen. Gjør dette hver gang, før hver kontrakt.

- Installer Cursor (cursor.com) og 1Password-appen. Be Lise om tilgang til hvelvet «EiraNova». Åpne Cursor og skriv «hent eiranova-platform», deretter «Sett opp tilganger» – Cursor ber om seks nøkler fra 1Password, én om gangen. Det tar fem minutter og gjøres én gang.
- Åpne mappen docs/ og les i denne rekkefølgen: README.md (oversikt), docs/PROCESS.md (metoden), docs/ROADMAP.md (planen), docs/contracts/CONTRACT_QUEUE.json (køen — hva som er gjort og planlagt).
- Logg inn på Vercel og Supabase med selskapets konto og bare se deg rundt. Ikke endre noe.

Del 4 — Din første kontrakt, steg for steg

Eksempel: endre teksten under «For eldre» på eiranova.no.

1. Last ned fersk zip av dev-branchen (Del 3). Start ny samtale i Claude-prosjektet EiraNova, last opp zip-filen, og lim inn utfylt endringsønske-mal: «Lag en Cursor-kontrakt for å endre teksten under «For eldre» på eiranova.no til: [ny tekst].»
2. Claude svarer med en ferdig kontrakt (K-MARKETING-0xx). Les den. Stemmer den med det dere ville? Hvis ikke: si fra i samme samtale, og Claude justerer.
3. Åpne Cursor, åpne repoet, start en ny chat. Lim inn kontrakten fra Claude. Cursor jobber, og kan spørre om bekreftelse før den kjører kommandoer — les hva den vil gjøre, og svar ja hvis det er innenfor kontrakten.
4. Når Cursor er ferdig, kopier hele rapporten dens og lim den inn i Claude-samtalen. Claude bekrefter at kontrakten er oppfylt — eller peker på hva som mangler, som dere gir tilbake til Cursor.
5. Åpne PR-en i GitHub (lenken står i rapporten). Under «Checks» finner du forhåndsvisningen (Vercel Preview). Se på endringen på mobil og PC.
6. Riktig? Skriv til Cursor: «Godkjent. Fullfør kjeden til main.» Feil? Beskriv det til Claude, som formulerer rettelsen til Cursor.
7. Etter noen minutter: sjekk eiranova.no. Endringen er live. Be Cursor oppdatere kontraktkøen (status merged) — det holder arkivet sant.

Første gang tar dette en time. Tredje gang tar det ti minutter. Og fordi Claude husker samtalen, kan dere spørre den om hva som helst underveis — «hva betyr dette?», «er det farlig?», «hva skjer hvis vi godkjenner?».

Del 5 — Slik beskriver dere et ønske til Claude

Bruk malen i docs/pilot/endringsønske-mal.md — kopier den inn i Claude-samtalen og fyll ut de ni punktene. Det er den raskeste veien til en presis kontrakt.

Jo mer konkret ønsket er, jo bedre blir kontrakten. En god beskrivelse har fire deler:

- **Hva:** «Vi vil at ... » — beskriv hva kunden eller dere skal se eller kunne gjøre.
- **Hvor:** hvilken app eller side (eiranova.no, oppstart-appen, kunde-appen ...), gjerne med skjermbilde.
- **Nøyaktig innhold:** nye tekster, priser eller navn skrevet ordrett slik de skal stå.
- **Hva som ikke skal endres:** «alt annet på siden skal være som før».

Eksempel: «Lag en Cursor-kontrakt. På eiranova.no, under Priser, skal tabellen vise: Praktisk hjelp 690 kr/time, Avlastning 690 kr/time, Følge til avtaler 590 kr/time. Introduksjonspris første besøk: 490 kr. Småteksten under tabellen skal si at prisene er inkl. mva. Ikke rør noe annet på siden. Vedlagt fersk zip.»

Claude lager da kontrakten etter denne malen — dere trenger ikke skrive den selv, men det er nyttig å kjenne den igjen:

K-[OMRÅDE]-[NUMMER] – [kort tittel]

- App: (marketing / oppstart / kunde-app / nurse-app / admin-app)
- Branch: feature/K-[ID] → PR → dev → PR → main
- Omfang: hvilke filer eller sider som skal endres. Ingenting annet.
- Autonomi: «Forhåndsgodkjent: branch, filendringer, commits, push, PR-kjede. Aldri direkte til dev/main, aldri force-push, aldri utenfor omfang, ingen hemmeligheter i filer.»
- Endringer: nummerert liste. Vær konkret: hvilken tekst skal bort, hvilken skal inn – ordrett.
- Utenfor scope: det Cursor ikke skal røre.
- Verifikasjon: hva som skal sjekkes før PR.
- Rapporter tilbake: endringsliste, verifikasjonsresultat, PR-numre.

Rapporten Cursor gir dere til slutt, limes inn i Claude-samtalen. Da lukkes sirkelen: den som skrev kontrakten, sjekker at den ble fulgt.

Egnede kontrakter å gjøre selv i starten: tekst- og prisendringer på eiranova.no · nye tjenester eller endrede beskrivelser i tjenestekatalogen · nye oppgaver eller seksjoner i oppstart-appen · e-postmaler (tekst) · bilder og logo · personvernerklæringen.

Del 6 — Trappen: hva dere gjør selv, hva dere får sjekket, hva dere bestiller

Den billigste måten å bygge på er: dere bygger, en konsulent kvalitetssikrer. En utvikler som skal lage noe fakturerer 20–40 timer; en som skal gjennomgå en ferdig PR fakturerer 2–4. Derfor er alt delt i tre nivåer.

Nivå 1 — dere gjør det selv, ingen QA nødvendig:

- Tekst, priser og bilder på eiranova.no
- Tjenester og beskrivelser i tjenestekatalogen
- Oppgaver og seksjoner i oppstart-appen
- E-postmaler (tekst), personvernerklæringen

Nivå 2 — dere gjør det selv, med veiledning første gang, og QA før prod:

- Koble en skjerm i appene til databasen (f.eks. «Mine oppdrag» viser ekte data)
- Ny e-postvarsling (f.eks. bekreftelse når et oppdrag er tildelt)
- Endringer på admin-sider (lister, filtre, felter)
- Slik: dere beskriver ønsket til Claude med fersk zip, Claude lager kontrakten (første gang med Richard som sparringspartner på beskrivelsen), Cursor bygger, rapporten går tilbake til Claude, og en konsulent gjennomgår PR-en i 2–4 timer før dere merger. Etter én gjennomført nivå 2-kontrakt gjør dere neste alene.

Nivå 3 — bestilles fra utvikler, og QA er obligatorisk:

- Innlogging, roller og tilgangsstyring
- Betaling (Vipps, Stripe)
- Databasens struktur og sikkerhetsregler (RLS)
- Integrasjoner (Tripletex, journalsystem)
- Cursor kan gjerne lage første utkast også her — men en fagperson skal lese og godkjenne før det når ekte kunder. Her er QA ikke sparing, det er forsikring.

Regelen som binder det sammen: **ingenting som berører innlogging, penger eller persondata går i produksjon uten at et menneske med fagkompetanse — ikke bare Cursor — har lest det.** Alt annet eier dere fullt ut.

En god bestilling til utvikler eller QA-konsulent er nesten det samme som en kontrakt: hva kunden skal oppleve (brukerhistorie), hva som er innenfor og utenfor, hva som skal fungere når det er ferdig (akseptansekriterier), og at arbeidet følger repoets metode (docs/PROCESS.md, kontrakt i køen, PR-kjede). Roadmapen (docs/ROADMAP.md) beskriver de neste kontraktene — gi den direkte til utvikleren. For QA: send lenken til PR-en og be om skriftlig tilbakemelding i PR-en, slik at vurderingen blir liggende i arkivet.

Viktig for eierskap: det dere betaler for, eier selskapet. Be alltid om at kode leveres inn i selskapets repo, aldri i utviklerens eget.

Del 7 — Når noe går galt

- Siden er nede eller viser feil etter en endring: Vercel → prosjektet → «Instant Roll-back» tar dere tilbake til forrige versjon på ett minutt. Gjør det først, finn feilen etterpå.
- Kontaktskjemaet sender ikke: sjekk Resend → Logs. Sjekk at API-nøkkelen finnes i Vercel → Environment Variables.
- Noen kommer ikke inn: aldri endre i Supabase Authentication på egen hånd — det er en utvikler-jobb.
- Usikre på noe som helst: lim det inn i Claude-samtalen og spør. Claude forklarer hva det betyr og hva dere bør gjøre — det er alltid det første steget.
- Cursor gjorde noe uventet: ikke godkjenn PR-en. Skriv «Avbryt, forklar hva du gjorde» og les svaret. En PR som ikke er godkjent, har ikke rørt noe som er live.
- Data i Supabase: se, ikke endre, med mindre dere vet nøyaktig hva dere gjør. Database har daglig sikkerhetskopi, men gjenoppretting er en utvikler-jobb.

Del 8 — Ordliste

- **Repo:** arkivet med all kode og dokumentasjon.
- **Zip av repoet:** en nedlastet kopi av arkivet slik det er akkurat nå. Claude trenger den for å lage kontrakter som treffer.
- **Claude-prosjekt:** en arbeidsflate på claude.ai der bakgrunnsdokumentene ligger fast, slik at hver ny samtale starter med riktig kontekst.
- **Branch:** en arbeidskopi der en endring lages uten å påvirke det som er live. main = live, dev = staging, feature/... = én kontrakt.
- **PR (pull request):** et forslag om å ta en endring inn. Kan ses, kommenteres og godkjennes før den blir live.
- **Preview:** automatisk testversjon av en PR på egen adresse.

- **Deploy:** å legge en ny versjon ut på nett. Vercel gjør det automatisk når main endres.
- **Kontrakt (K-...):** en presis bestilling. Én om gangen.
- **Discovery (D-...):** noe uventet som ble oppdaget underveis, logget for læring.
- **Env-variabel:** en hemmelig innstilling (nøkkel) som ligger i Vercel, aldri i koden.
- **RLS:** sikkerhetsregler i databasen som bestemmer hvem som får se hva. Rør aldri.

Del 9 — Køen på GitHub: ideer som må vente

Mange gode ideer kan ikke gjøres nå — de venter på org.nummer, på penger, på at noe annet er ferdig, eller bare på tid. Regelen er enkel: en idé som er verdt å huske, blir til en kontrakt i køen med én gang. Da er den ikke lenger avhengig av at noen husker den, og en utvikler kan plukke den opp om et halvt år uten å spørre hva dere mente.

Hvor køen ligger. To filer i repoet, begge synlige i nettleseren på github.com/eiranova-no/eiranova-platform:

- `docs/contracts/CONTRACT_QUEUE.json` — selve køen. Én oppføring per kontrakt med ID, tittel, status, mål og avhengigheter. Den er skrevet for maskiner, men fullt lesbar.
- `docs/status/CONTROL_CENTER.md` — den samme køen presentert som en oversikt for mennesker: hva som er ferdig, hva som pågår, hva som venter og hvorfor. Åpne denne først.
- `docs/contracts/active/` — spesifikasjonen (hele kontrakten) for det som er klart eller pågår.
- `docs/contracts/DISCOVERIES.json` — loggen over uventede funn (D-numre): ting som ble oppdaget underveis og som påvirker senere arbeid. Les den før dere bestiller noe stort.

Statusene betyr dette:

- **merged** — ferdig og live
- **active** — pågår akkurat nå (det kan bare være én om gangen — det er hovedregelen i hele metoden)
- **ready** — kontrakten er skrevet og klar til å startes
- **planned** — idéen er beskrevet, spesifikasjonen kan mangle, og den venter på tur
- **blocked** — kan ikke startes før noe annet er på plass (grunnen står i `blocked_reason`)
- **paused / superseded** — satt på vent, eller erstattet av en annen kontrakt

Slik parkerer dere en idé:

1. Skriv til Claude i EiraNova-prosjektet: «Vi har en idé som må vente: [beskriv]. Den kan ikke gjøres før [grunn]. Lag kontrakt og legg den i køen.» Legg ved fersk zip.
2. Claude skriver kontrakten, foreslår ID og status (`planned` eller `blocked` med grunn), plasserer den i riktig fase i roadmapen, og lager en liten Cursor-instruks som legger den i køen.
3. Dere gir instruks til Cursor. Den oppdaterer `CONTRACT_QUEUE.json`, regenererer `CONTROL_CENTER.md` og legger det inn i arkivet. Ingen kode berøres — dette er en ren «governance»-endring.
4. Idéen er nå synlig på GitHub for alle, med begrunnelse for hvorfor den venter. Når grunnen er borte, ber dere Claude flytte den til `ready` — og så starter det vanlige løpet.

Hverdagsrutinen — slik fortsetter dere arbeidet, hver gang:

- **Vanlig dag:** åpne Cursor i repoet og skriv «Hva er neste i køen?». Cursor leser `CONTROL_QUEUE.json` og viser den første kontrakten som er ready og ikke blokkert. Svar «Start den» — så begynner det vanlige løpet. Cursor viser hva som står først; den vurderer ikke om rekkefølgen er riktig.
- **Når noe har endret seg** — penger, tid, en ny kunde, en ny idé — eller dere er usikre på rekkefølgen: last ned fersk zip, åpne Claude-prosjektet og skriv «Her er repoet. Hva bør vi prioritere nå, og hvorfor?». Claude ser køen, roadmapen, discoveries og det som faktisk er bygget, og gir et begrunnet forslag.
- **Tommelfingerregel:** Cursor for «hva står først», Claude for «hva bør stå først».

Manuelt oppslag fungerer også: åpne `CONTROL_CENTER.md` på GitHub, se på det som er ready og planned, sjekk avhengighetene, og velg den som gir mest verdi og ikke er blokkert.

Hvorfor dette er viktig for dere som eiere: køen er selskapets minne. Den viser hva som er bygget, hva som er bestemt, og hva som er tenkt. Hold den sann, og den jobber for dere.

Del 10 — Opplæringsøktene (med Richard)

- **Økt 1 — Kart og kompass (2 t):** Del 1, 3 og 9. Begge logger inn overalt, inkludert Claude-prosjektet. Lager zip av repoet sammen, og åpner `CONTROL_CENTER.md` på GitHub for å lese køen. Sjekkliste: GitHub-bruker, Claude-prosjekt åpnet, Cursor installert, repo klonet, **Sett opp tilganger fullført – Cursor sa Ferdig**, zip lastet opp én gang, køen lest og forstått.
- **Økt 2 — Ønske til kode (2 t):** Del 2, 4 og 5. Hver av dere gjennomfører én ekte liten kontrakt: ønske til Claude → kontrakt → Cursor → rapport tilbake til Claude → forhåndsvisning → live. Sjekkliste: én PR laget, rapport verifisert av Claude, forhåndsvist, godkjent, live, kø oppdatert.
- **Økt 3 — Drift og feil (2 t):** Del 7. Gjør en rollback med vilje og se at det virker. Gå gjennom overleveringsprotokollen: hvem eier hva, hvor ligger nøklene. Sjekkliste: rollback utført, hvelv gjennomgått.
- **Økt 4 — Trappen, køen og bestilling (1 t):** Del 6, 9 og roadmapen. Parker sammen én fremtidig idé som kontrakt i køen, og skriv én QA-bestilling til konsulent. Sjekkliste: én kontrakt lagt i kø via Claude og Cursor, QA-bestilling skrevet, avtalt hvem som er QA-konsulent.

Etter økt 4 er dere selvgående. Alt dere lurte på senere, står i docs/ og i køen — og denne håndboken kan dere bruke til å lære opp neste person.